

# CSE327

# SOFTWARE ENGINEERING

AKM Iqtidar Newaz [IqN]

Lecture – 17



# WHAT IS REGRESSION TESTING?

- **Definition:**

Regression testing verifies that recent code changes (bug fixes, enhancements, or new features) haven't adversely affected the existing functionality.

- **Purpose:**

Maintain software quality and ensure stability across continuous updates.



# WHERE IS REGRESSION TESTING USED?

## ■ **Development Lifecycle:**

- ✓ Performed after bug fixes, refactoring, or adding new features.
- ✓ Integrate into continuous integration (CI) pipelines for automated test runs.

## ■ **Application Domains:**

- ✓ Critical systems like banking, healthcare, and e-commerce.
- ✓ Large-scale software (operating systems, cloud services) where even minor regressions can be costly.

## ■ **Benefits:**

- ✓ Rapid detection of issues.
- ✓ Ensuring that legacy features remain functional after updates.
- ✓ Increased developer confidence during iterative development.



# TOOLS FOR REGRESSION TESTING IN C++

- **Google Test (gtest):**
  - ✓ Widely used, feature-rich testing framework.
  - ✓ Supports assertions, fixtures, and parameterized tests.
- **CppUnit:**
  - ✓ A port of JUnit for C++, useful for those familiar with Java testing practices.
- **Boost.Test:**
  - ✓ Part of the Boost library suite; integrates well with projects already using Boost.
- **Catch2:**
  - ✓ A modern, header-only testing framework that is easy to integrate and use.
- **Continuous Integration Tools:**
  - ✓ Jenkins, Travis CI, GitLab CI/CD, and others to automatically run regression tests after every code commit.



# C++ CODE EXAMPLE: REGRESSION TESTING WITH GOOGLE TEST

```
#include <vector>
#include <string>

struct Item {
    std::string name;
    double price;
    int quantity;
};

double calculateTotal(const std::vector<Item>& cart, double discount = 0.0) {
    double total = 0;
    for (const auto& item : cart) {
        total += item.price * item.quantity;
    }
    double discountAmount = total * discount / 100;
    return total - discountAmount;
}
```



# C++ CODE EXAMPLE: REGRESSION TESTING WITH GOOGLE TEST

```
#include <gtest/gtest.h>
#include <vector>
#include <string>

// Reuse the Item structure and calculateTotal function from above

TEST(CalculateTotalTest, WithoutDiscount) {
    std::vector<Item> cart = {
        {"Item A", 100, 2},
        {"Item B", 50, 1}
    };
    // Expected total: 100*2 + 50*1 = 250
    EXPECT_DOUBLE_EQ(calculateTotal(cart), 250.0);
}

TEST(CalculateTotalTest, WithDiscount) {
    std::vector<Item> cart = {
        {"Item A", 100, 2},
        {"Item B", 50, 1}
    };
    // Expected total with 10% discount: 250 - 25 = 225
    EXPECT_DOUBLE_EQ(calculateTotal(cart, 10), 225.0);
}

int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

## Output:

```
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from CalculateTotalTest
[ RUN    ] CalculateTotalTest.WithoutDiscount
[      OK ] CalculateTotalTest.WithoutDiscount (0 ms)
[ RUN    ] CalculateTotalTest.WithDiscount
[      OK ] CalculateTotalTest.WithDiscount (0 ms)
[-----] 2 tests from CalculateTotalTest (0 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (0 ms total)
[ PASSED ] 2 tests.
```



# CODE EXAMPLE: OOP-BASED SHOPPING CART

```
#include <gtest/gtest.h>
#include <vector>
#include <string>

// Class representing an individual item.
class Item {
public:
    std::string name;
    double price;
    int quantity;

    // Constructor initializes the item's properties.
    Item(const std::string &n, double p, int q)
        : name(n), price(p), quantity(q) {}
};
```

```
// Class representing a shopping cart.
class ShoppingCart {
private:
    std::vector<Item> items;

public:
    // Add an item to the cart.
    void addItem(const Item &item) {
        items.push_back(item);
    }

    // Calculate the total cost of all items in the cart.
    double calculateTotal() const {
        double total = 0;
        for (const auto &item : items) {
            total += item.price * item.quantity;
        }
        return total;
    }

    // Apply a discount (in percentage) to the total.
    double applyDiscount(double discountPercent) const {
        double total = calculateTotal();
        double discountAmount = total * discountPercent / 100;
        return total - discountAmount;
    }
};
```



# CODE EXAMPLE: OOP-BASED SHOPPING CART

```
// Google Test Cases
// Test that verifies the total calculation without discount.
TEST(ShoppingCartTest, TotalCalculation) {
    ShoppingCart cart;
    cart.addItem(Item("Apple", 2.5, 4));    // 4 * 2.5 = 10
    cart.addItem(Item("Banana", 1.2, 5));    // 5 * 1.2 = 6
    // Expected total: 10 + 6 = 16
    EXPECT_DOUBLE_EQ(cart.calculateTotal(), 16.0);
}

// Test that verifies discount application.
TEST(ShoppingCartTest, DiscountApplication) {
    ShoppingCart cart;
    cart.addItem(Item("Laptop", 1000, 1));    // 1000
    cart.addItem(Item("Mouse", 50, 2));        // 2 * 50 = 100
    // Total = 1100, and with a 10% discount, expected value = 1100 - 110 = 990
    EXPECT_DOUBLE_EQ(cart.applyDiscount(10), 990.0);
}

// Test the behavior of an empty cart.
TEST(ShoppingCartTest, EmptyCart) {
    ShoppingCart cart;
    EXPECT_DOUBLE_EQ(cart.calculateTotal(), 0.0);
    // Discount on 0 should remain 0.
    EXPECT_DOUBLE_EQ(cart.applyDiscount(10), 0.0);
}
```



# CODE EXAMPLE: OOP-BASED SHOPPING CART

```
// Main function that initializes Google Test and runs all tests.
int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}
```

## Output:

```
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from ShoppingCartTest
[ RUN      ] ShoppingCartTest.TotalCalculation
[          OK ] ShoppingCartTest.TotalCalculation (0 ms)
[ RUN      ] ShoppingCartTest.DiscountApplication
[          OK ] ShoppingCartTest.DiscountApplication (0 ms)
[ RUN      ] ShoppingCartTest.EmptyCart
[          OK ] ShoppingCartTest.EmptyCart (0 ms)
[-----] 3 tests from ShoppingCartTest (0 ms total)
[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (0 ms total)
[ PASSED   ] 3 tests.
```

